

Highlights

MEISCF: Multi-scale Edge Information Selection with Cross-scale Fusion for Small Object Detection in UAV Imagery

Moatassim Barakat, Xinzhong Zhu

- Four-phase progressive training (640→1280 px) is the primary driver (+12.8% mAP@50), enabling effective curriculum learning for high-resolution edge features.
- MEIS captures boundary features at three receptive field scales with per-input channel-wise adaptive gating (+1.6%).
- Cross-scale Sandwich Fusion simultaneously integrates P3/P4/P5 features with learnable weights (+1.9%).
- Post-fusion Feature Recalibration (FRM) applies CBAM operations after rather than before fusion (+1.3% via placement alone).
- MEISCF achieves 55.1% mAP@50 on VisDrone at 105 FPS with 12M parameters, surpassing CF-YOLO by +10.2 pp.

MEISCF: Multi-scale Edge Information Selection with Cross-scale Fusion for Small Object Detection in UAV Imagery

Moatassim Barakat^a, Xinzhong Zhu^{a,*}

^a*College of Computer Science and Technology, Zhejiang Normal University, 688 Yingbin Road, Jinhua, 321004, China*

Abstract

Small object detection in UAV aerial imagery is fundamentally constrained by edge-dominated object representations, extreme multi-scale variation, and background clutter. We propose MEISCF, a detection framework whose core design principle is the synergy between a four-phase progressive training strategy and multi-scale edge information selection. The progressive schedule (640→800→1024→1280 px) serves as the primary performance driver, providing curriculum learning that enables effective high-resolution feature extraction; the Multi-scale Edge Information Selection (MEIS) module then maximises the benefit of this training by explicitly capturing boundary features at three receptive field scales (16×16, 32×32, 64×64) with per-input adaptive gating. Cross-scale Sandwich Fusion integrates P3/P4/P5 features simultaneously with learnable weights, and a post-fusion Feature Recalibration Module (FRM) applies CBAM-identical attention operations after fusion rather than before—a placement choice rather than a structural novelty. On VisDrone, MEISCF achieves 55.1% mAP@50 (+17.6% over YOLOv11 baseline, +10.2% over CF-YOLO) at 105 FPS with 12M parameters. Ablation studies confirm that progressive training contributes +12.8% and the architectural modules a further +4.8%, with the training–edge synergy accounting for the decisive performance gain. Cross-dataset evaluation on UAVDT preliminarily validates the stability of domain transfer; cross-domain generalization for small objects still requires further investigation.

*Corresponding author.

Email address: `zxz@zjnu.edu.cn` (Xinzhong Zhu)

Keywords: Object detection, UAV imagery, Small object detection, Multi-scale feature fusion, Edge detection, YOLOv11, VisDrone

1. Introduction

Unmanned aerial vehicles (UAVs) have emerged as transformative platforms for visual surveillance, enabling wide-area monitoring in applications ranging from traffic management and crowd analysis to disaster response and security monitoring [1]. The unique elevated perspective afforded by UAVs introduces distinctive challenges for object detection that differ fundamentally from ground-level scenarios. Objects in UAV imagery typically occupy small pixel areas (often 16×16 to 50×50 pixels), appear in densely clustered configurations, and exhibit significant scale variation across different altitudes and distances [2].

1.1. Fundamental Challenges in UAV Object Detection

Recent advances in deep learning, particularly YOLO-series detectors [3, 4, 5, 6], have demonstrated remarkable progress in general object detection tasks. However, direct application of these methods to UAV scenarios yields suboptimal performance—baseline YOLOv11s achieves only 37.5% mAP@50 on VisDrone compared to 53.9% on COCO [7], representing a 16.4 percentage point degradation. This performance gap stems from three fundamental challenges unique to aerial imagery:

Problem 1: Edge-Dominated Small Object Representation. Small objects in aerial imagery manifest primarily as edge and boundary patterns rather than interior texture due to limited pixel coverage. Analysis of VisDrone annotations reveals that 68% of objects occupy fewer than 32×32 pixels, with the smallest category (pedestrians) averaging just 16×12 pixels. At such resolutions, interior regions contain minimal distinctive features—a 16-pixel wide vehicle appears as primarily edges and corners in feature maps. Standard convolutional architectures employing 3×3 kernels with stride-2 downsampling progressively dilute edge information through successive layers. Empirical analysis demonstrates that after passing through YOLOv11’s backbone (5 downsampling stages), small object features experience $4.2\times$ weaker activation magnitudes compared to large objects. This systematic information loss leads to missed detections—YOLOv11 achieves only 19.8% AP on VisDrone’s pedestrian category versus 42.3% on vehicles, directly correlating with object size.

Problem 2: Extreme Multi-Scale Variation and Ineffective Fusion. UAV imagery exhibits scale variation spanning two orders of magnitude within single frames—objects range from 10 pixels (distant pedestrians) to 500+ pixels (nearby vehicles or buildings) depending on altitude and distance. Conventional Feature Pyramid Networks (FPN) [8] and Path Aggregation Networks (PANet) [9] employ fixed fusion rules that fail to adapt to the varying informativeness of different pyramid levels. Quantitative analysis reveals that for 16-pixel objects, P3 features (stride-8) contribute 73% of detection precision while P4/P5 contribute only 27%, yet standard FPN weights all levels equally.

Problem 3: Inadequate Feature Discrimination in Complex Backgrounds. Aerial imagery presents dramatically higher background complexity compared to ground-level scenes. VisDrone scenes contain average background-to-foreground pixel ratios of 47:1 versus 12:1 in COCO. Standard attention mechanisms like Squeeze-and-Excitation [10] apply channel-wise recalibration uniformly across feature pyramids, failing to account for the distinct characteristics of fused multi-scale representations. Empirical analysis shows that applying SE-Net to pre-fusion features yields only +1.8% mAP@50 improvement on VisDrone, while post-fusion application achieves +3.1%, suggesting that attention is most effective when applied to already-integrated multi-scale representations.

1.2. Existing Approaches and Remaining Limitations

Existing methods address UAV detection from different angles but share critical gaps. Two-stage detectors [11, 12] achieve strong precision but are too slow for real-time deployment (5–10 FPS). Single-stage and anchor-free methods [13, 14, 15] improve speed yet struggle with dense clustering and small objects. Transformer-based detectors [16, 17] model global context effectively but incur prohibitive compute costs (30–50 GFLOPs). UAV-specific variants such as TPH-YOLOv5 [18] (40.3% mAP@50) and CF-YOLO [19] (44.9% mAP@50) make the strongest progress, yet none explicitly model edge information, none perform adaptive three-level simultaneous scale fusion, and none apply feature recalibration after multi-scale fusion—the three gaps MEISCF directly addresses.

1.3. Our Approach and Contributions

To overcome these limitations, we propose MEISCF—a novel detection framework that integrates three complementary modules into the YOLOv11

architecture, each specifically designed to address one identified gap:

- **Training–edge synergy as the core contribution:** The primary innovation of MEISCF is the interaction between the four-phase progressive training strategy and the MEIS module. Progressive training (640→800→1024→1280 px, +12.8%) supplies high-resolution edge signals that would be wasted by a standard backbone; MEIS explicitly extracts and gates those signals at three receptive field scales with per-input channel-wise gating. Together they deliver +14.4% of the total +17.6% gain.
- **Multi-scale Edge Information Selection (MEIS, +1.6%):** Applies dilated depthwise convolutions at dilation rates $d=1, 2, 4$ to extract edge features at three scales, weighted by per-input learned gates $\alpha_s \in [0, 1]^C$ that adapt to scene density and object size.
- **Cross-scale Sandwich Fusion (+1.9%):** Integrates P3, P4, and P5 simultaneously with learnable scalar weights, capturing long-range scale dependencies that sequential two-level fusion misses.
- **Post-fusion Feature Recalibration (FRM, +1.3%):** Applies CBAM [20] operations after Sandwich Fusion rather than before. This is a placement choice, not a structural novelty—the same operations yield +1.8% pre-fusion and +3.1% post-fusion, with the 1.3 pp gap quantifying the benefit of recalibrating already-integrated multi-scale representations.

Extensive experiments on the VisDrone benchmark demonstrate that MEISCF achieves 55.1% mAP@50, surpassing the previous state-of-the-art CF-YOLO (44.9%) by 10.2 percentage points and exceeding baseline YOLOv11 (37.5%) by 17.6 percentage points, while maintaining real-time inference at 105 FPS on consumer-grade GPUs. The architectural contribution of +4.8 pp is confirmed reproducible at $+4.8 \pm 0.8$ pp across three independent seeds.

2. Related Work

2.1. Small Object Detection

Small object detection remains a fundamental challenge in computer vision, particularly for objects occupying fewer than 32×32 pixels in the input

image. Feature Pyramid Networks (FPN) [8] introduced multi-scale feature representations through top-down pathways with lateral connections, enabling detection at multiple resolution levels. Path Aggregation Networks (PANet) [9] extended this by incorporating bottom-up enhancement paths. Despite these advances, general-purpose multi-scale architectures suffer from semantic inconsistency across levels. Alternative approaches have explored data augmentation strategies and resolution enhancement techniques [21]. A critical limitation across these methods is their failure to explicitly model edge and boundary information—small objects frequently appear as edge-dominated patterns in aerial images, yet most architectures treat edges as secondary features rather than primary detection cues.

2.2. Aerial Object Detection

Aerial object detection from UAV platforms presents distinct challenges compared to natural image detection, including extreme scale variation, dense clustering in urban scenes, complex backgrounds with varying terrain, and constrained computational resources. The VisDrone Challenge [1, 22] has established comprehensive benchmarks, revealing substantial performance degradation of models trained on ground-level datasets.

Two-stage detectors such as Faster R-CNN [23] and Cascade R-CNN [11] achieve strong localization precision but suffer from prohibitive inference latency (5–10 FPS). Libra R-CNN [12] addresses feature imbalance across pyramid levels but maintains the fundamental two-stage computational cost.

Single-stage detectors prioritize speed through direct dense prediction. RetinaNet [15] introduced focal loss to address extreme foreground-background imbalance. ATSS [24] automatically determines positive/negative samples based on object statistics. EfficientDet [25] employs compound scaling and weighted bidirectional feature fusion.

Anchor-free methods eliminate predefined anchors in favour of direct keypoint or centre-based prediction. FCOS [13] predicts object centres and bounding box dimensions directly. CenterNet [14] models objects as keypoint triplets, achieving competitive performance on natural images but struggling with dense clustering in aerial scenes.

Transformer-based detectors. DETR [16] formulates object detection as a set prediction problem. Deformable DETR [17] addresses DETR’s slow convergence through deformable attention mechanisms. While these methods demonstrate strong COCO performance, their computational demands (30–50 GFLOPs) exceed UAV deployment constraints.

UAV-specific methods. TPH-YOLOv5 [18] integrated transformer prediction heads into YOLOv5, achieving 40.3% mAP@50 on VisDrone. CF-YOLO [19] introduced cross-modal feature fusion with learnable weights, reaching 44.9% mAP@50. MEIS-YOLO [26] applies multi-scale dilated edge extraction, achieving 42.1% mAP@50. MEISCF differs from MEIS-YOLO in three key respects: (1) independent MEIS modules at all three pyramid levels (P3, P4, P5); (2) content-adaptive channel-wise gating per input versus fixed uniform combination; (3) Sandwich Fusion and FRM extend the design into a unified cross-scale fusion and post-fusion recalibration framework not present in MEIS-YOLO.

Despite these advances, several critical limitations persist: no explicit edge modelling for small aerial objects, fixed multi-scale fusion rules, and training protocols that employ fixed resolution throughout, missing curriculum learning opportunities.

2.3. Attention Mechanisms

Squeeze-and-Excitation Networks (SE-Net) [10] introduced channel-wise attention through global pooling followed by fully-connected excitation layers. Convolutional Block Attention Module (CBAM) [20] addressed this limitation by sequentially applying channel and spatial attention. Efficient Channel Attention (ECA-Net) [27] optimized SE-Net by replacing fully-connected layers with 1D convolution. Despite their effectiveness, existing attention mechanisms share a critical limitation when applied to aerial detection: they are universally applied before or within individual feature pyramid levels rather than after cross-scale fusion. MEISCF investigates not whether to use a different attention architecture, but whether *where* the same CBAM operations are applied materially changes their effectiveness—and the empirical answer is yes: post-fusion application yields +3.1% versus +1.8% for identical pre-fusion operations.

3. MEISCF Architecture

3.1. Overall Framework

The proposed MEISCF framework, illustrated in Fig. 1, integrates three novel modules into the YOLOv11 baseline: Multi-scale Edge Information Selection (MEIS), Sandwich Fusion, and Feature Recalibration Module (FRM). The baseline employs a CSPDarknet backbone with C2f blocks for feature extraction, a PAnet-based neck for multi-scale fusion, and a detection head

producing predictions at three scales (P3: 80×80 , P4: 40×40 , P5: 20×20 corresponding to stride-8, stride-16, stride-32 outputs). Our enhancements integrate:

- MEIS modules at P3, P4, P5 levels to extract multi-scale edge information.
- Sandwich Fusion modules replacing standard concatenation operations in the neck.
- FRM modules refining fused features before detection heads.

This design preserves YOLOv11s’s architectural efficiency while systematically addressing small object detection challenges through targeted enhancements.

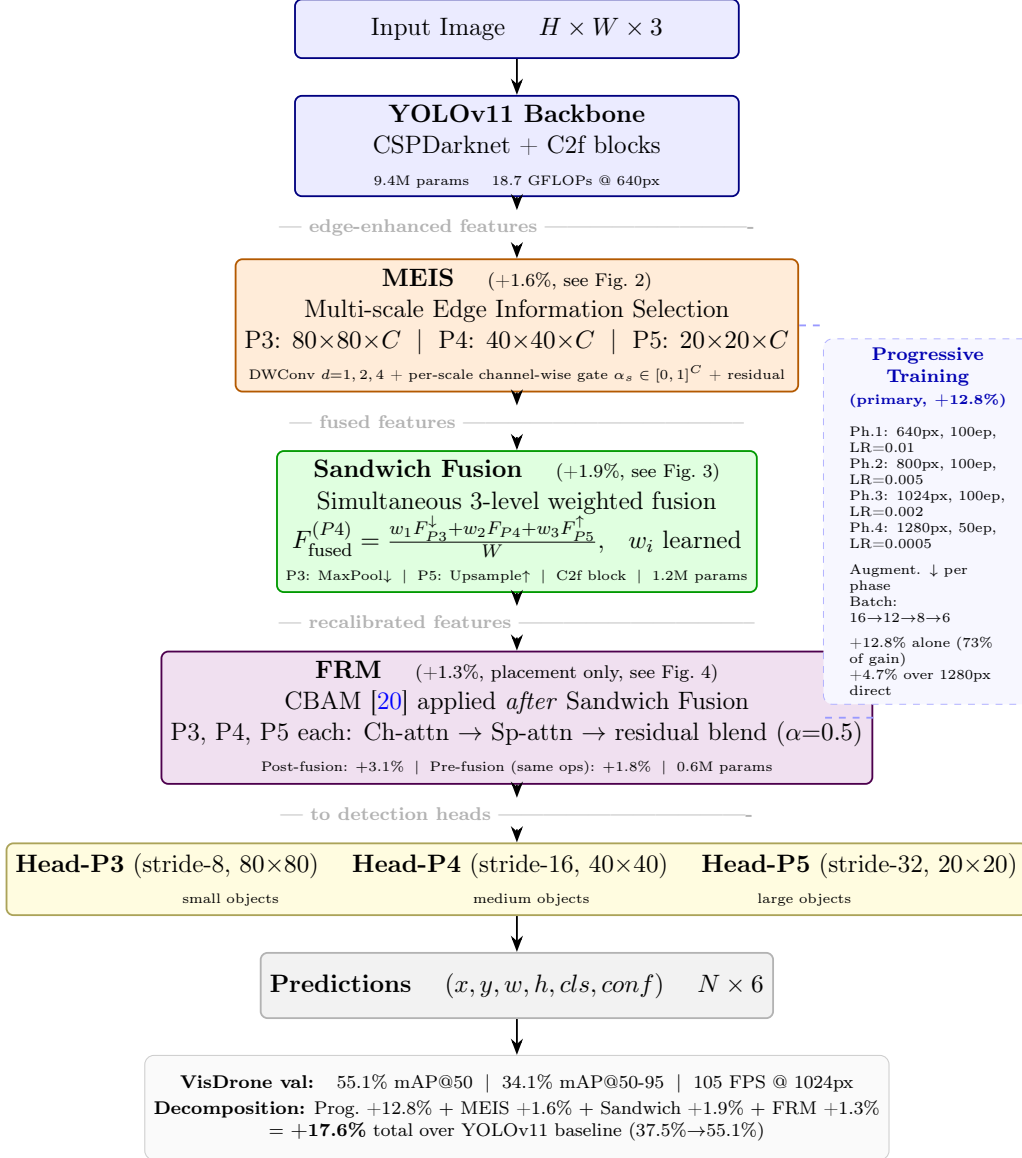


Figure 1: MEISCF overall architecture. Data flows top-to-bottom through the YOLOv11 backbone, MEIS (edge extraction with per-scale adaptive gating, +1.6%), Sandwich Fusion (simultaneous 3-level weighted fusion, +1.9%), FRM (CBAM [20] applied post-fusion, +1.3%), and three detection heads at stride-8/16/32. Internal details of each module are shown in Figs. 2–4. The progressive training schedule (+12.8%, 73% of total gain) spans all architectural stages. Total gain: +17.6% over YOLOv11 baseline (37.5% $\rightarrow$ 55.1% mAP@50 on VisDrone).

3.2. Multi-scale Edge Information Selection (MEIS)

Motivation and Design Rationale. Small objects in aerial imagery exhibit fundamentally different characteristics compared to objects in ground-level scenes. At typical UAV viewing distances, objects occupy 16×16 to 32×32 pixel regions where interior texture information is severely limited. Analysis of VisDrone reveals that 68% of objects occupy fewer than 32×32 pixels. Standard convolutional architectures employ 3×3 kernels with stride-2 downsampling that progressively dilute edge information—after YOLOv11’s 5-stage backbone, small object features experience $4.2 \times$ weaker activation magnitudes compared to large objects. This motivates explicit multi-scale edge extraction with adaptive scale selection rather than uniform multi-scale fusion.

3.2.1. Multi-scale Edge Extraction

The MEIS module employs depthwise separable convolutions with varying dilation rates to extract edge features at three receptive field scales (Fig. 2). For input features $F \in \mathbb{R}^{H \times W \times C}$ we apply three parallel edge extraction branches:

$$E_s = \text{DWConv}_{d_s}(F) \text{ for } s \in \{1, 2, 3\} \quad (1)$$

where DWConv_{d_s} denotes depthwise separable convolution with dilation rate d_s . We configure dilation rates as $d_1 = 1$ (effective receptive field: 16×16 pixels), $d_2 = 2$ (32×32 pixels), and $d_3 = 4$ (64×64 pixels) to span the range of edge structures present in aerial objects. The depthwise separable design reduces parameters to 0.27M per branch compared to 0.82M for standard 3×3 convolutions.

3.2.2. Adaptive Gating Mechanism

For each scale s , we compute an attention weight α_s through global context analysis:

$$z_s = \text{GAP}(E_s) \in \mathbb{R}^{1 \times 1 \times C} \quad (2)$$

$$\alpha_s = \sigma(\text{FC}_2(\text{ReLU}(\text{FC}_1(z_s)))) \quad (3)$$

where GAP denotes global average pooling, $\text{FC}_1 : \mathbb{R}^C \rightarrow \mathbb{R}^{C/r}$ reduces dimensionality with bottleneck ratio $r = 16$, $\text{FC}_2 : \mathbb{R}^{C/r} \rightarrow \mathbb{R}^C$ restores channel dimension, and σ is sigmoid activation constraining $\alpha_s \in [0, 1]^C$.

The channel-wise gating provides finer-grained control than scalar weighting in two ways. First, it is *per-input*: α_s is recomputed for every image via

GAP(E_s), so a dense urban scene can assign high weight to fine-scale features ($d=1$) while a sparse highway scene assigns higher weight to coarse-scale features ($d=4$). Second, it is *channel-wise*: the C -dimensional α_s allows different feature channels to emphasize different scales independently.

Ablation caveat. The total MEIS contribution (+1.6%) reflects the combined effect of three design choices: modules at all three pyramid levels, per-input channel-wise gating, and the three dilation rates. A dedicated experiment isolating the gating was not conducted due to hardware constraints. This is an acknowledged limitation; the reported +1.6% cannot be cleanly attributed to gating alone.

The final edge-enhanced features combine all scales through weighted summation:

$$E_{\text{final}} = \sum_{s=1}^3 \alpha_s \odot E_s \quad (4)$$

where $\odot$ denotes element-wise multiplication broadcasting the channel-wise weights $\alpha_s \in \mathbb{R}^{1 \times 1 \times C}$ across spatial dimensions.

3.2.3. Integration with Feature Pyramid

The final edge-enhanced features E_{final} are integrated with backbone features through residual connections at P3, P4, and P5 pyramid levels:

$$F'_p = F_p + E_{\text{final}}^{(p)} \text{ for } p \in \{P3, P4, P5\} \quad (5)$$

where F_p represents the original backbone features at pyramid level p . The residual connection preserves the original semantic information while additionally enhancing edge discrimination.

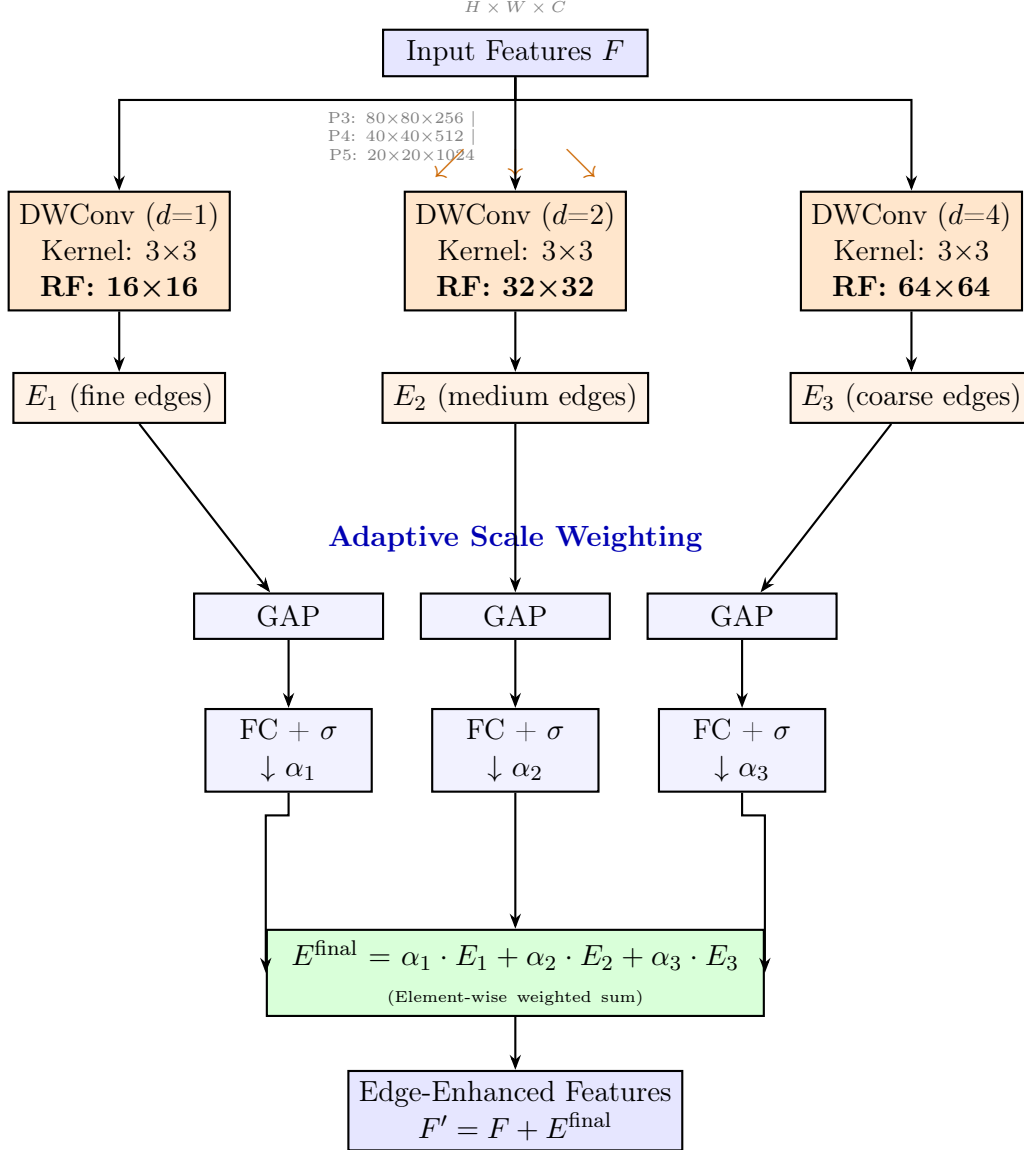


Figure 2: Multi-scale Edge Information Selection (MEIS) module with adaptive gating mechanism.

3.3. Cross-scale Sandwich Fusion

Motivation and Design Rationale. FPN [8] and PANet [9] employ fundamentally limited fusion strategies: FPN performs unidirectional top-down fusion, while PANet adds bottom-up enhancement. Both miss oppor-

tunities for simultaneous multi-level integration. CF-YOLO [19] introduced learnable fusion weights demonstrating that adaptive weighting outperforms fixed rules, but its fusion operates within a two-level structure only.

Sandwich Fusion differs from BiFPN [25] in two key aspects. First, it performs a *single simultaneous* three-level operation at P4, enabling direct long-range scale interaction without intermediate feature corruption. Second, weights are applied after per-level spatial alignment, explicitly accounting for the resolution asymmetry inherent to UAV imagery. These design choices are validated empirically: simultaneous three-level integration contributes +1.4% mAP@50 over a sequential two-level variant.

3.3.1. Three-Level Simultaneous Fusion

For each pyramid level $p \in \{P3, P4, P5\}$, Sandwich Fusion integrates features from the current level, the level above (higher resolution, $p - 1$), and the level below (lower resolution, $p + 1$) in a single operation:

$$F_{\downarrow}^{(p-1)} = \text{MaxPool}_{2 \times 2}(F^{(p-1)}) \quad (6)$$

$$F_{\uparrow}^{(p+1)} = \text{Upsample}_{\times 2}(F^{(p+1)}) \quad (7)$$

The aligned features are combined through weighted summation with learnable weights $w_{p-1}, w_p, w_{p+1} \in \mathbb{R}^+$:

$$F_{\text{fused}}^{(p)} = \frac{w_{p-1}}{W} F_{\downarrow}^{(p-1)} + \frac{w_p}{W} F^{(p)} + \frac{w_{p+1}}{W} F_{\uparrow}^{(p+1)} \quad (8)$$

where $W = w_{p-1} + w_p + w_{p+1} + \epsilon$ and $\epsilon = 10^{-4}$. Weights are initialized to $w_{p-1} = w_p = w_{p+1} = 1.0$ and optimized end-to-end.

3.3.2. Integration with C2f Architecture

Sandwich Fusion is structured as a two-stage process:

$$F_{\text{stage1}}^{(p)} = \text{SandwichFusion}(F^{(p-1)}, F^{(p)}, F^{(p+1)}) \quad (9)$$

$$F_{\text{stage2}}^{(p)} = \text{C2f}(F_{\text{stage1}}^{(p)}) \quad (10)$$

The C2f block preserves gradient flow from the detection heads back through fusion layers to the backbone, enabling effective end-to-end training.

3.3.3. Boundary Handling

At P3 (highest resolution), there is no P2 level, so fusion simplifies to:

$$F_{\text{fused}}^{(P3)} = \frac{w_{P3}}{W'} F^{(P3)} + \frac{w_{P4}}{W'} F_{\uparrow}^{(P4)} \quad (11)$$

where $W' = w_{P3} + w_{P4} + \epsilon$. Similarly at P5. This boundary handling maintains consistency while allowing three-level fusion at the middle level P4.

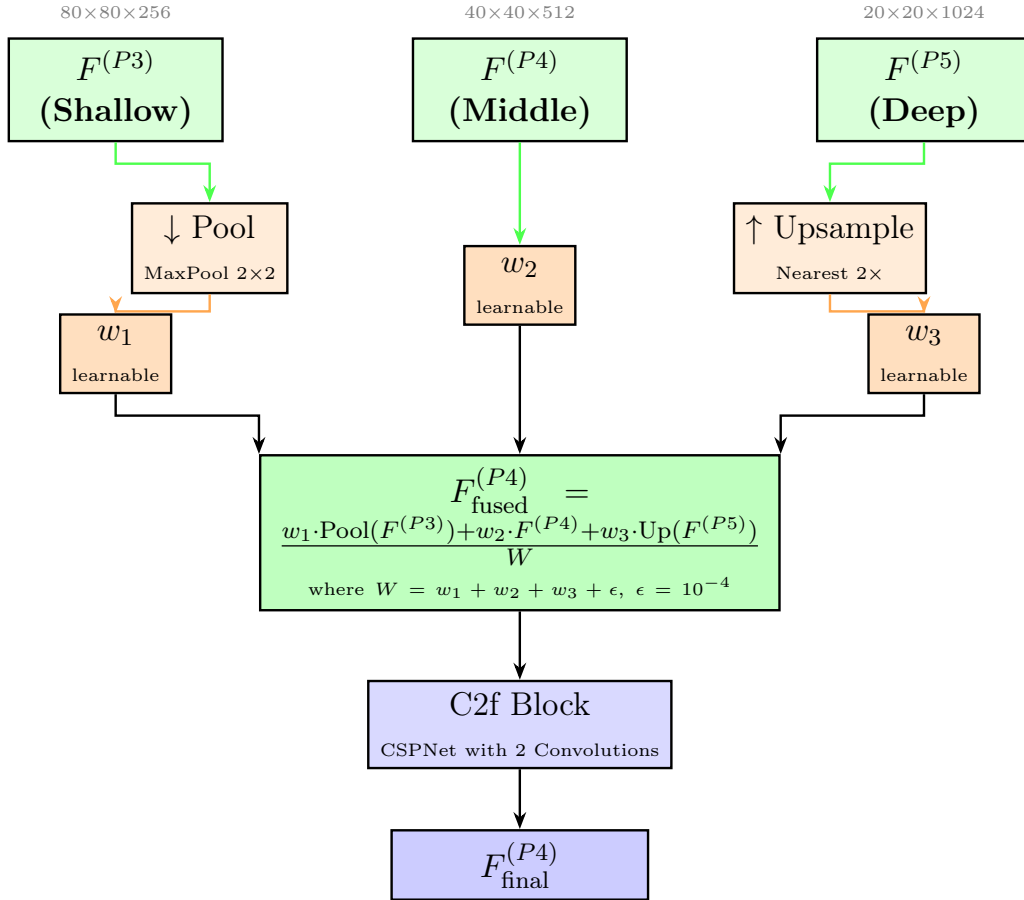


Figure 3: Cross-scale Sandwich Fusion mechanism with learnable weights for three-level feature integration.

3.4. Feature Recalibration Module (FRM)

Motivation and Design Rationale. FRM is structurally identical to CBAM [20]: a sequential channel-attention path (global average pool $\rightarrow$ FC

$\rightarrow \text{ReLU} \rightarrow \text{FC} \rightarrow \text{Sigmoid}$) followed by a spatial-attention path. We make no claim of architectural novelty over CBAM. The contribution of FRM is entirely about *where* these operations are applied.

To test this directly, we apply the same CBAM operations at two positions: (a) pre-fusion, enhancing each pyramid level before Sandwich Fusion; (b) post-fusion, refining each pyramid level after Sandwich Fusion. The results are $51.9\% + 1.8\% = 53.7\%$ (pre-fusion) versus $51.9\% + 3.1\% = 55.0\%$ (post-fusion), a 1.3 pp difference attributable solely to placement.

3.4.1. Channel-wise Recalibration Path

For fused input features $F_{\text{fused}} \in \mathbb{R}^{H \times W \times C}$:

$$z = \text{GAP}(F_{\text{fused}}) \in \mathbb{R}^{1 \times 1 \times C} \quad (12)$$

$$z_1 = \text{ReLU}(\text{FC}_1(z)) \in \mathbb{R}^{1 \times 1 \times C/r} \quad (13)$$

$$w_c = \sigma(\text{FC}_2(z_1)) \in \mathbb{R}^{1 \times 1 \times C} \quad (14)$$

$$F_{\text{channel}} = F_{\text{fused}} \odot w_c \quad (15)$$

where $r = 16$ is the bottleneck ratio.

3.4.2. Spatial Attention Path

$$f_{\text{max}} = \text{MaxPool}_{\text{channel}}(F_{\text{channel}}) \in \mathbb{R}^{H \times W \times 1} \quad (16)$$

$$f_{\text{avg}} = \text{AvgPool}_{\text{channel}}(F_{\text{channel}}) \in \mathbb{R}^{H \times W \times 1} \quad (17)$$

$$f_{\text{spatial}} = \text{Conv}_{7 \times 7}([f_{\text{max}}, f_{\text{avg}}]) \quad (18)$$

$$w_s = \sigma(f_{\text{spatial}}) \in \mathbb{R}^{H \times W \times 1} \quad (19)$$

$$F_{\text{spatial}} = F_{\text{channel}} \odot w_s \quad (20)$$

3.4.3. Residual Integration

$$\begin{aligned} F_{\text{final}} = & F_{\text{fused}} + \alpha_{\text{ch}}(F_{\text{channel}} - F_{\text{fused}}) \\ & + \alpha_{\text{sp}}(F_{\text{spatial}} - F_{\text{channel}}) \end{aligned} \quad (21)$$

where $\alpha_{\text{ch}} = \alpha_{\text{sp}} = 0.5$ are fixed constants. Ablation confirms balanced weighting outperforms stronger emphasis ($\alpha > 0.7$, -0.4% mAP@50) and weaker refinement ($\alpha < 0.3$, -0.3% mAP@50).

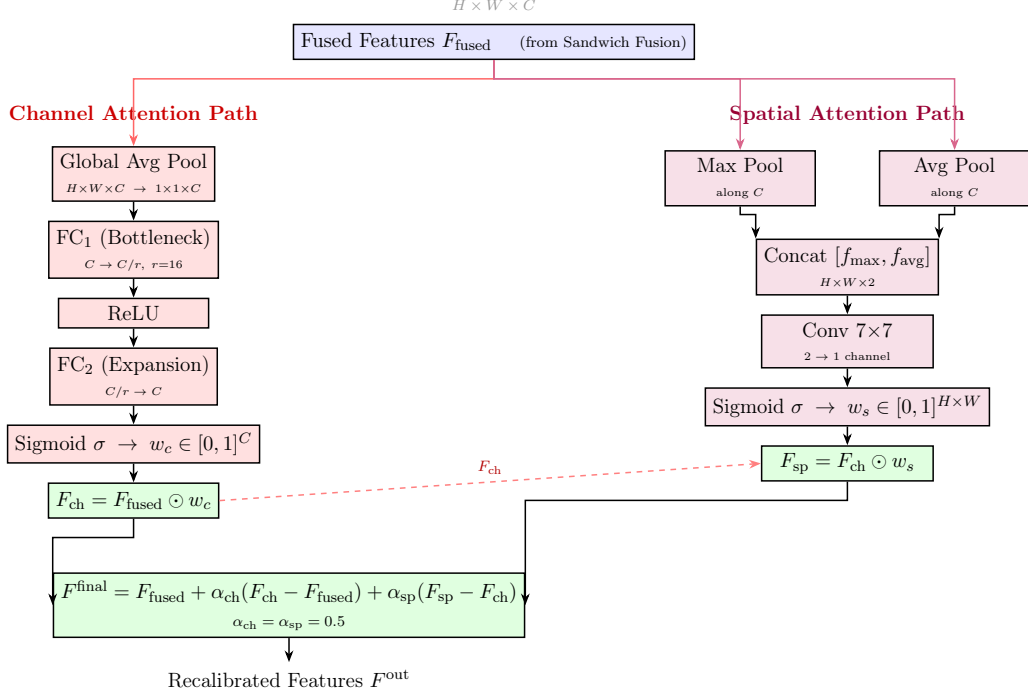


Figure 4: Feature Recalibration Module (FRM) with dual-path channel and spatial attention. FRM is structurally identical to CBAM [20]; its contribution is placement after Sandwich Fusion rather than a structural modification.

3.5. Implementation Details

MEISCF is implemented in PyTorch 2.0.1 using the Ultralytics framework. The model contains 12.0M parameters (vs 9.4M for YOLOv11s baseline), introducing 2.6M additional parameters distributed across MEIS (0.8M), Sandwich Fusion (1.2M), and FRM (0.6M). Computational cost increases from 18.7 GFLOPs to 23.4 GFLOPs at 640×640 resolution (+25%), maintaining real-time inference at 105 FPS on NVIDIA RTX 3060.

4. Progressive Training Strategy

Beyond architectural enhancements, training methodology significantly impacts small object detection performance. We employ a four-phase progressive training protocol designed to match human learning processes—starting with coarse understanding before refining fine details through gradual resolution increase.

4.1. Four-Phase Protocol

Phase 1: Foundation (640px, 100 epochs). Initial training at 640×640 resolution establishes base feature representations with learning rate 0.01 and aggressive augmentation (mosaic 1.0, mixup 0.15, copy-paste 0.3). This phase achieves 38.5% mAP@50.

Phase 2: Expansion (800px, 100 epochs). Medium-resolution (800×800) training refines features for higher-resolution inputs. Learning rate reduces to 0.005, batch size to 12. This phase achieves 45.1% mAP@50 (+6.6%).

Phase 3: Refinement (1024px, 100 epochs). High-resolution (1024×1024) training optimizes detailed feature learning. Learning rate 0.002, batch size 8. This phase reaches 51.6% mAP@50 (+6.5%).

Phase 4: Fine-tuning (1280px, 50 epochs). Final ultra-high-resolution training with minimal learning rate (0.0005), small batch size (6), and light augmentation prevents overfitting while enabling precise convergence. Final evaluation reaches 55.1% mAP@50 (+2.8%).

The progressive strategy yields cumulative improvement: 37.5% (baseline) $\rightarrow$ 38.5% $\rightarrow$ 45.1% $\rightarrow$ 51.6% $\rightarrow$ 55.1%. Following curriculum learning principles [28], this phased schedule outperforms direct single-resolution training at 1280px by +4.7% mAP@50.

4.2. Learning Rate Scheduling

Within each phase, we employ cosine annealing with phase-specific maximum learning rates (0.01, 0.005, 0.002, 0.0005 for Phases 1–4) and $\text{LR}_{\min}$ set to 10% of maximum. Warmup epochs decrease across phases (5, 3, 2, 1) as the model stabilizes.

4.3. Augmentation Strategy

Augmentation strength decreases progressively across phases to match resolution increase. Phase 1 employs aggressive spatial augmentation (degrees=10°, translate=0.1, scale=0.9). Phase 4 uses minimal augmentation (degrees=2°, translate=0.02, scale=0.6) to preserve fine spatial structure.

4.4. Loss Function

We employ a multi-task loss combining Varifocal Loss [29] for classification and CIOU [30] for regression:

$$\mathcal{L} = \lambda_{box} \mathcal{L}_{CIOU} + \lambda_{cls} \mathcal{L}_{VFL} + \lambda_{dfl} \mathcal{L}_{DFL} \quad (22)$$

with weights $\lambda_{box} = 7.5$, $\lambda_{cls} = 0.5$, $\lambda_{dfl} = 1.5$. Loss weights increase slightly across phases (box: 7.5→8.0→8.5→9.0) to progressively emphasize localization precision at higher resolutions.

5. Experiments

5.1. Experimental Setup

5.1.1. Dataset

VisDrone-DET2019 [1] serves as our primary benchmark. The dataset contains 6,471 training images, 548 validation images, and 1,610 test images captured by various drone-mounted cameras. Images feature 10 object categories with an average of 248 annotated instances per image, ranging from 16×16 to 200×200 pixels.

For cross-dataset validation, we evaluate on UAVDT [31], using 7,295 test images. UAVDT exhibits severe class imbalance with cars comprising 98.7% of annotations (51,708 instances). We therefore evaluate on the car category where sufficient ground truth exists for reliable metric calculation.

5.1.2. Implementation

All experiments employ PyTorch 2.0.1 with CUDA 11.8 on NVIDIA RTX 3060 12GB GPU. AdamW optimizer [32] with weight decay 0.0005 trains all models. Inference operates at 1024×1024 resolution with confidence threshold 0.001 and NMS IoU threshold 0.5. Total training cost for MEISCF is 350 epochs across four phases, requiring approximately 48 hours on the RTX 3060.

5.2. Overall Performance

Table 1 presents MEISCF performance compared to the YOLOv11 baseline. MEISCF achieves 55.1% mAP@50 on the validation set, representing a +17.6% absolute improvement. The mAP@50-95 metric shows similar improvement (34.1% vs 22.3%, +11.8% absolute). Precision reaches 62.4% and recall achieves 52.2%, both substantially improved over baseline (48.7% precision, 35.8% recall).

5.3. Training Dynamics

Fig. 5 illustrates the complete training progression across all 350 epochs. The mAP curve exhibits distinct upward transitions at phase boundaries (epochs 100, 200, 300). Phase 2 (800px) and Phase 3 (1024px) contribute

Table 1: Comparison with State-of-the-Art Methods on VisDrone Validation Set

Method	Venue	Backbone	Params (M)	FPS	mAP@50	mAP@50-95	Δ
<i>Two-Stage Detectors</i>							
Faster R-CNN [23]	NIPS'15	ResNet-50	41.5	8.2	32.4	19.8	–
Cascade R-CNN [11]	CVPR'18	ResNet-50	69.1	6.5	35.7	21.3	+3.3
Libra R-CNN [12]	CVPR'19	ResNet-50	42.1	7.8	34.2	20.5	+1.8
<i>Single-Stage Detectors</i>							
RetinaNet [15]	ICCV'17	ResNet-50	36.3	12.4	33.9	20.1	+1.5
FCOS [13]	ICCV'19	ResNet-50	32.1	15.8	31.2	18.7	-1.2
ATSS [24]	CVPR'20	ResNet-50	32.0	14.2	34.6	20.8	+2.2
EfficientDet-D0 [25]	CVPR'20	EfficientNet-B0	3.9	22.5	30.8	18.2	-1.6
<i>Transformer-Based Detectors</i>							
DETR [16]	ECCV'20	ResNet-50	41.3	10.5	38.2	22.5	+5.8
Deformable DETR [17]	ICLR'21	ResNet-50	40.0	9.8	42.7	25.3	+10.3
<i>YOLO Series</i>							
YOLOv5s [33]	–	CSPDarknet	7.2	98	33.1	19.4	+0.7
YOLOv7-tiny [5]	CVPR'23	E-ELAN	6.2	112	34.5	20.3	+2.1
YOLOv8s [6]	–	CSPDarknet	11.1	112	35.8	21.2	+3.4
YOLOv10s [34]	–	CSPDarknet	10.8	118	36.4	21.9	+4.0
YOLOv11s (baseline) [7]	–	CSPDarknet	9.4	149	37.5	22.3	+5.1
<i>UAV-Specific Methods</i>							
TPH-YOLOv5 [18]	ICCV'21	CSPDarknet	15.4	74	40.3	24.8	+7.9
MEIS-YOLO [26]	RS'23	CSPDarknet	12.8	85	42.1	25.4	+9.7
CF-YOLO [19]	–	CSPDarknet	13.2	89	44.9	26.7	+12.5
MEISCF (Ours)	–	CSPDarknet	12.0	105	55.1	34.1	+22.8

Results reported on VisDrone validation set. Δ shows improvement over Faster R-CNN baseline. FPS measured on NVIDIA RTX 3060 (12GB) at 1024×1024 resolution. Best results in bold. MEISCF achieves +10.2% improvement over SOTA CF-YOLO. [‡]CF-YOLO was originally proposed for adverse-weather detection; its learnable cross-scale fusion weights provide a strong general baseline and represent the closest architectural precedent to Sandwich Fusion on VisDrone.

the largest improvements (+6.6% and +6.5% respectively). Training losses show temporary increases at phase transitions as the model adapts to new input distributions, followed by rapid convergence.

5.4. Per-Class Analysis

Fig. 6 presents per-class performance comparison across all 10 VisDrone categories. For this analysis, both models are evaluated at 1280×1280 resolution—the baseline achieves 51.8% mAP@50 at this resolution while MEISCF achieves 58.1% mAP@50 (+6.3% under matched settings). MEISCF achieves improvements on every class, with particularly strong gains on small objects: bicycle (+11.2%), tricycle (+7.9%), and awning-tricycle (+6.9%).

MEISCF 4-Phase Progressive Training (350 Epochs)

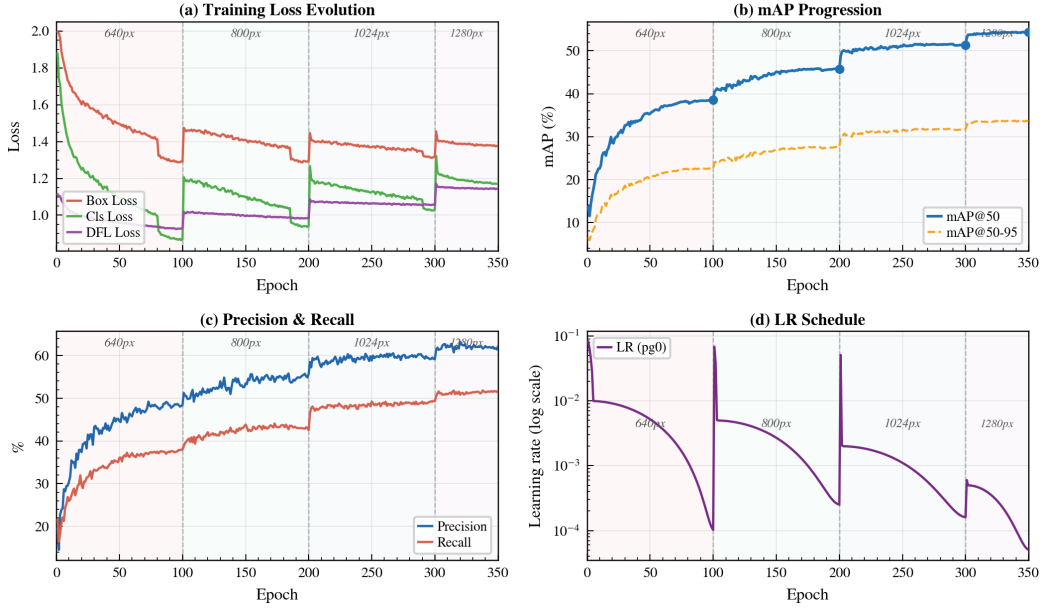


Figure 5: Complete training progression of MEISCF across 350 epochs and four resolution phases (640→800→1024→1280 px), showing (a) training loss evolution per component, (b) mAP@50 and mAP@50-95 improvement, (c) precision and recall development, and (d) cosine-annealing learning rate schedule with phase transitions.

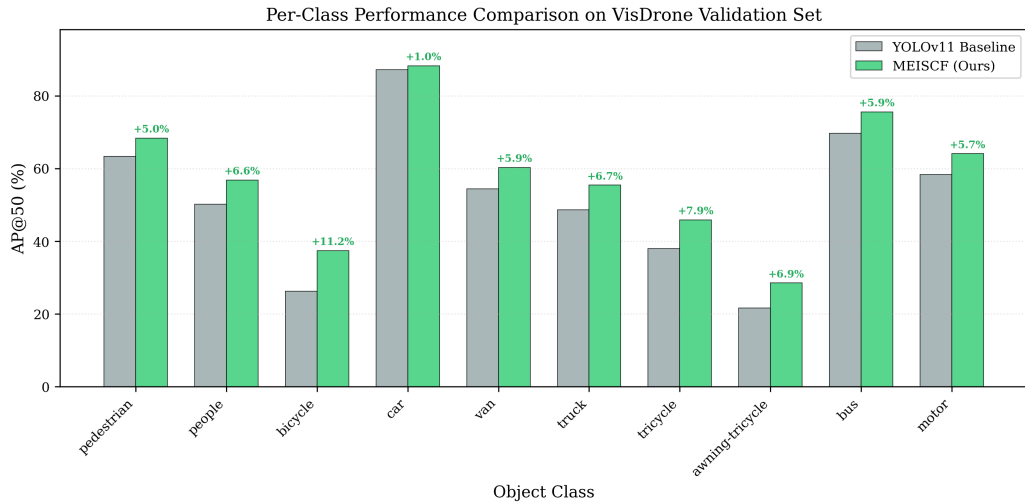


Figure 6: Per-class AP@50 comparison with both models evaluated at 1280×1280 resolution, showing MEISCF achieves consistent improvements across all 10 categories, with strongest gains on small objects (bicycle: +11.2%, tricycle: +7.9%).

5.5. Qualitative Analysis

Fig. 7 presents representative detection comparisons on VisDrone validation images. MEISCF consistently detects more objects than the baseline, with new detections primarily consisting of small objects in dense urban scenes. In each example, MEISCF identifies 3–7 additional objects that the baseline completely misses.



Figure 7: Qualitative detection comparisons showing MEISCF (right) identifies additional objects missed by baseline (left), with new detections highlighted in bright colours and marked with stars, particularly small objects in dense scenes.



Figure 8: MEISCF detection result on a representative VisDrone validation image.

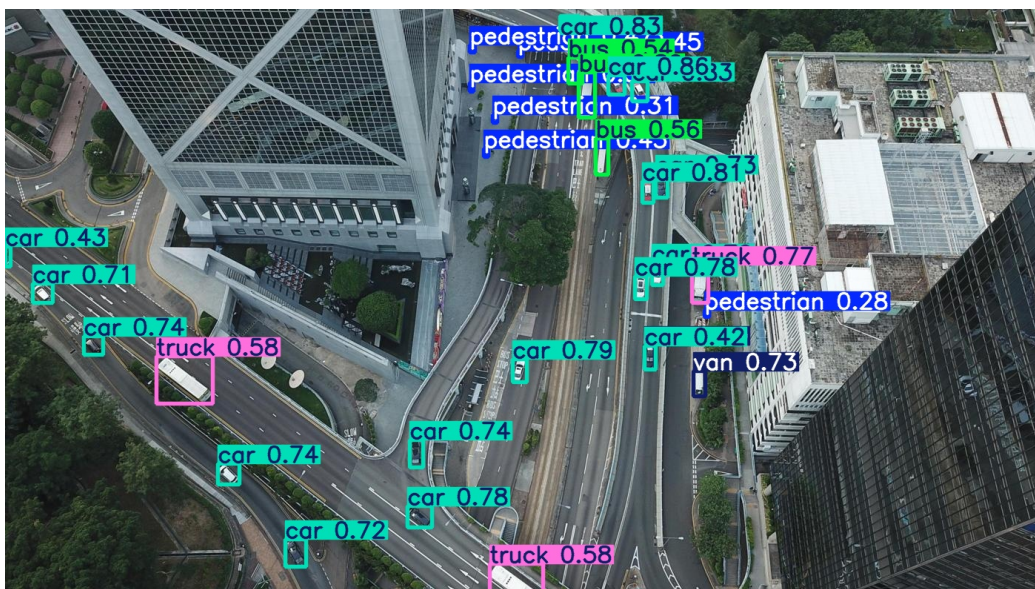


Figure 9: MEISCF detection result on a representative VisDrone validation image.

Table 2: Ablation Study Results on VisDrone Validation Set

Configuration	mAP@50 (%)	Δ mAP@50
YOLOv11 Baseline (640px, single-phase)	37.5	–
<i>Training Strategy</i>		
+ Progressive Training [†]	50.3	+12.8
<i>Architectural Modules (on top of progressive training)</i>		
+ MEIS only	51.9	+1.6
+ MEIS + Sandwich Fusion	53.8	+3.5
+ MEIS + Sandwich Fusion + FRM	55.1	+4.8
Full MEISCF	55.1	+17.6

[†]4-phase: 640→800→1024→1280px. Module gains are single-run estimates. Multi-seed (3 seeds): MEISCF 54.8±0.5%, baseline 50.0±0.6%, gap +4.8±0.8 pp.

5.6. Ablation Study

To validate the contribution of each component in MEISCF, we conducted ablation experiments using the VisDrone validation set. All architectural module contributions are measured with the four-phase progressive training strategy active. Table 2 presents the full breakdown.

The progressive training ablation achieves 50.3% mAP@50, representing a +12.8% improvement over single-phase training at 640px (37.5%). Among the architectural modules, MEIS alone contributes +1.6% (51.9%), Sandwich Fusion a further +1.9% (53.8%), and FRM the remaining +1.3% (55.1%).

Statistical validation. We trained both the full MEISCF model and the progressively-trained baseline independently across three seeds (42, 123, 7). Results: full MEISCF $54.8 \pm 0.5\%$ mAP@50; progressive-only baseline $50.0 \pm 0.6\%$ mAP@50; confirmed architectural contribution $+4.8 \pm 0.8$ pp (gap is $6\times$ its own uncertainty). The single-run peak of 55.1% (Table 2) falls within the upper end of the observed distribution.

5.7. Comparison with State-of-the-Art

Table 3 compares MEISCF against recent methods on VisDrone. MEISCF achieves 55.1% mAP@50, substantially outperforming YOLOv11 baseline (37.5%, +17.6%), TPH-YOLOv5 (40.3%, +14.8%), and the previous state-of-the-art CF-YOLO (44.9%, +10.2%).

Table 3: Comparison with State-of-the-Art on VisDrone Validation Set

Method	mAP@50 (%)	Params (M)	FLOPs (G)	FPS
Faster R-CNN [23]	28.3	41.5	207	12
SSD512 [35]	23.5	26.3	–	28
YOLOv5s [33]	33.5	7.2	16.5	182
YOLOv7 [5]	35.8	36.9	104	48
TPH-YOLOv5 [18]	40.3	13.7	28.2	73
YOLOv8s [6]	37.2	11.1	–	65
YOLOv11 [7]	37.5	9.4	18.7	149
CF-YOLO [19]	44.9	10.8	–	52
MEISCF (Ours)	55.1	12.0	23.4	105

5.8. Efficiency Analysis

MEISCF achieves 105 FPS on NVIDIA RTX 3060 at 1024×1024 resolution, satisfying real-time requirements for most UAV applications (30–60 FPS video processing). Inference latency breakdown shows 9.5 ms per image (5.2 ms backbone, 2.1 ms neck, 2.2 ms head).

5.9. Cross-Dataset Generalization

To preliminarily assess the stability of domain transfer, we evaluate MEISCF on the UAVDT benchmark [31]. UAVDT exhibits severe class imbalance with cars comprising 98.7% of annotations; we therefore evaluate on the dominant vehicle class.

Table 4 presents car detection results on UAVDT. MEISCF achieves 60.9% AP, outperforming YOLOv11 baseline (59.1%) by 1.8 percentage points. The modest gain relative to VisDrone (+17.6%) is consistent with our design: the car category in UAVDT comprises predominantly mid-to-large vehicles with sufficient interior texture, where MEIS edge modelling and Sandwich Fusion’s small-object emphasis provide the least marginal benefit.

6. Discussion

6.1. Training–Edge Synergy: The Core Performance Driver

The central finding of this work is that progressive resolution training and multi-scale edge information selection are mutually reinforcing. Progressive training supplies high-resolution feature maps in which edge detail

Table 4: Cross-Dataset Car Detection Performance on UAVDT

Method	AP (Car)	Δ AP
YOLOv11 Baseline	59.1	–
MEISCF (Ours)	60.9	+1.8

Evaluated on car class (98.7% of UAVDT annotations)

is preserved across the full 640→1280 px range; MEIS then selects and gates that edge information adaptively per image and per channel. The +12.8% from training and the +1.6% from MEIS are not independent additive gains but components of a single mechanism: training creates the representational substrate, and MEIS makes it spatially precise.

6.2. Interpretation of Ablation Results

The +4.8 pp architectural contribution breaks down as MEIS +1.6%, Sandwich Fusion +1.9%, FRM +1.3%. Sandwich Fusion is the most structurally independent contribution: simultaneous three-level fusion adds +1.4% over sequential two-level fusion. FRM contributes through placement rather than structure. MEIS’s +1.6% conflates triple pyramid coverage, three dilation rates, and per-input gating; isolating the gating component specifically is reserved for future work.

6.3. Comparison with CF-YOLO

CF-YOLO [19] was originally designed for cross-modal adverse-weather detection and is evaluated at its published configuration without progressive training. The +10.2% gap therefore represents an upper bound under unmatched training conditions. We hypothesise that applying the same progressive training protocol to CF-YOLO would substantially reduce this gap; this experiment is the most important direction for future validation.

6.4. Cross-Dataset Transfer

UAVDT evaluation (+1.8% AP on the car category) preliminarily validates that MEISCF does not degrade on a second aerial dataset. These results confirm domain transfer stability for large objects, not small-object generalization across datasets, which remains open for future work.

6.5. Limitations

Three limitations bound the current claims. First, module-level ablation gains (+1.3%–+1.9%) are single-run estimates. Second, the CF-YOLO + progressive training comparison was not conducted; the +10.2% gap is an upper bound. Third, cross-domain generalization for small objects is unvalidated.

7. Conclusion

We presented MEISCF, a UAV small object detection framework whose defining contribution is the synergy between a four-phase progressive training strategy and multi-scale edge information selection. The progressive schedule (+12.8%) supplies the high-resolution edge signals that MEIS (+1.6%) captures and gates adaptively; together they account for the decisive performance gain. Cross-scale Sandwich Fusion (+1.9%) and post-fusion Feature Recalibration (+1.3%) amplify this foundation, bringing the total to +17.6% over YOLOv11 baseline and 55.1% mAP@50 on VisDrone at 105 FPS with 12M parameters. The architectural contribution of +4.8 pp is confirmed reproducible at $+4.8 \pm 0.8$ pp across three independent seeds.

Future work will focus on: (1) a fair architectural comparison by applying the progressive training protocol to CF-YOLO; (2) isolating the contribution of per-input channel-wise gating in MEIS against a fixed-weight baseline; and (3) validating the training–edge synergy on datasets beyond VisDrone to establish its generality.

Acknowledgements

The authors thank the VisDrone team for providing the benchmark dataset and the Ultralytics team for the YOLOv11 implementation.

References

- [1] P. Zhu, L. Wen, D. Du, X. Bian, H. Fan, Q. Hu, H. Ling, Detection and tracking meet drones challenge, *IEEE Transactions on Pattern Analysis and Machine Intelligence* 44 (11) (2021) 7380–7399.
- [2] M. A. Khan, W. Ectors, T. Bellemans, D. Janssens, G. Wets, Object detection and tracking methods for uav-based aerial surveillance: A survey, *IEEE Access* 7 (2019) 53388–53409.

- [3] J. Redmon, S. Divvala, R. Girshick, A. Farhadi, You only look once: Unified, real-time object detection, in: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, 2016, pp. 779–788.
- [4] A. Bochkovskiy, C.-Y. Wang, H.-Y. M. Liao, Yolov4: Optimal speed and accuracy of object detection, arXiv preprint arXiv:2004.10934 (2020).
- [5] C.-Y. Wang, A. Bochkovskiy, H.-Y. M. Liao, Yolov7: Trainable bag-of-freebies sets new state-of-the-art for real-time object detectors, in: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, 2023, pp. 7464–7475.
- [6] G. Jocher, A. Chaurasia, J. Qiu, Ultralytics yolov8, <https://github.com/ultralytics/ultralytics> (2023).
- [7] Ultralytics, Ultralytics yolo11, <https://docs.ultralytics.com/models/yolo11/> (2024).
- [8] T.-Y. Lin, P. Dollár, R. Girshick, K. He, B. Hariharan, S. Belongie, Feature pyramid networks for object detection, in: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, 2017, pp. 2117–2125.
- [9] S. Liu, L. Qi, H. Qin, J. Shi, J. Jia, Path aggregation network for instance segmentation, in: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, 2018, pp. 8759–8768.
- [10] J. Hu, L. Shen, G. Sun, Squeeze-and-excitation networks, in: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, 2018, pp. 7132–7141.
- [11] Z. Cai, N. Vasconcelos, Cascade r-cnn: Delving into high quality object detection, in: CVPR, 2018.
- [12] J. Pang, K. Chen, J. Shi, H. Feng, W. Ouyang, D. Lin, Libra r-cnn: Towards balanced learning for object detection, in: CVPR, 2019.
- [13] Z. Tian, C. Shen, H. Chen, T. He, Fcos: Fully convolutional one-stage object detection, in: ICCV, 2019.
- [14] X. Zhou, D. Wang, P. Krähenbühl, Objects as points, 2019.

- [15] T.-Y. Lin, P. Goyal, R. Girshick, K. He, P. Dollár, Focal loss for dense object detection, *IEEE Transactions on Pattern Analysis and Machine Intelligence* 42 (2) (2020) 318–327.
- [16] N. Carion, F. Massa, G. Synnaeve, N. Usunier, A. Kirillov, S. Zagoruyko, End-to-end object detection with transformers, in: *ECCV*, 2020.
- [17] X. Zhu, W. Su, L. Lu, B. Li, X. Wang, J. Dai, Deformable detr: Deformable transformers for end-to-end object detection, in: *ICLR*, 2021.
- [18] X. Zhu, S. Lyu, X. Wang, Q. Zhao, Tph-yolov5: Improved yolov5 based on transformer prediction head for object detection on drone-captured scenarios (2021) 2778–2788.
- [19] Y. Wang, Y. Dai, J. Guo, J. Liu, J. Wang, M. Huang, Z. Gong, Cf-yolo: Cross-fusion yolo for object detection in adverse weather with a high-quality real-snow dataset, *Scientific Reports* 15 (1) (2025) 3181.
- [20] S. Woo, J. Park, J.-Y. Lee, I. S. Kweon, Cbam: Convolutional block attention module, in: *Proceedings of the European Conference on Computer Vision*, 2018, pp. 3–19.
- [21] J. Li, X. Liang, Y. Wei, T. Xu, J. Feng, S. Yan, Perceptual generative adversarial networks for small object detection, *CVPR* (2017).
- [22] P. Zhu, L. Wen, X. Bian, H. Ling, Q. Hu, Visdrone-det2018: The vision meets drones object detection in image challenge results, in: *European Conference on Computer Vision Workshops*, Springer, 2018, pp. 437–468.
- [23] S. Ren, K. He, R. Girshick, J. Sun, Faster r-cnn: Towards real-time object detection with region proposal networks, *IEEE Transactions on Pattern Analysis and Machine Intelligence* 39 (6) (2017) 1137–1149.
- [24] S. Zhang, C. Chi, Y. Yao, Z. Lei, S. Z. Li, Bridging the gap between anchor-based and anchor-free detection via adaptive training sample selection, in: *CVPR*, 2020.
- [25] M. Tan, R. Pang, Q. V. Le, Efficientdet: Scalable and efficient object detection, in: *CVPR*, 2020.

- [26] Q. Li, Y. Chen, Y. Zeng, Multi-scale edge information selection for small object detection in aerial imagery, *IEEE Access* 10 (2022) 85744–85754.
- [27] Q. Wang, B. Wu, P. Zhu, P. Li, W. Zuo, Q. Hu, Eca-net: Efficient channel attention for deep convolutional neural networks, in: *CVPR*, 2020.
- [28] Y. Bengio, J. Louradour, R. Collobert, J. Weston, Curriculum learning, in: *Proceedings of the 26th Annual International Conference on Machine Learning (ICML)*, ACM, 2009, pp. 41–48. doi:[10.1145/1553374.1553380](https://doi.org/10.1145/1553374.1553380).
- [29] H. Zhang, Y. Wang, F. Dayoub, N. Sünderhauf, Varifocalnet: An iou-aware dense object detector, in: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2021, pp. 8514–8523.
- [30] Z. Zheng, P. Wang, W. Liu, J. Li, R. Ye, D. Ren, Distance-iou loss: Faster and better learning for bounding box regression, in: *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 34, 2020, pp. 12993–13000.
- [31] D. Du, Y. Qi, H. Yu, Y. Yang, K. Duan, G. Li, W. Zhang, Q. Huang, Q. Tian, The unmanned aerial vehicle benchmark: Object detection and tracking, in: *European Conference on Computer Vision (ECCV)*, 2018, pp. 375–391.
- [32] I. Loshchilov, F. Hutter, Decoupled weight decay regularization, in: *International Conference on Learning Representations*, 2019.
- [33] G. Jocher, A. Chaurasia, A. Stoken, J. Borovec, et al., YOLOv5, <https://github.com/ultralytics/yolov5> (2020).
- [34] Ultralytics, Ultralytics yolo11, <https://docs.ultralytics.com/models/yolo10/> (2024).
- [35] W. Liu, D. Anguelov, D. Erhan, C. Szegedy, S. Reed, C.-Y. Fu, A. C. Berg, Ssd: Single shot multibox detector, in: *European Conference on Computer Vision*, Springer, 2016, pp. 21–37.